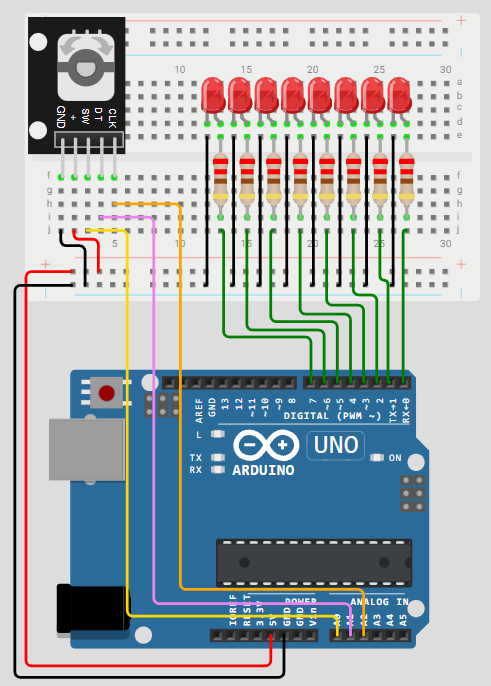
Atividade_02 - Revisão de Conceitos. Livro AVR e Arduino – Técnicas de Projeto
Capítulos: 4 (Manipulação de bits) e 5 (Portas)

Título: Conhecendo o Atmega328 e acessando portas de saída usando C/C++ / Debounce de botões e Encoders de posição

Objetivos: Entender a importância do debounce de botões e também o funcionamento de encoders como interface ao microcontrolador.

Nesta prática utilizaremos o Wokwi para simular um circuito simples usando o microcontrolador Atmega328p, utilizado nas placas Arduino UNO. Como entrada será utilizado um encoder rotativo e como saída 8 leds. O circuito base se encontra no link:
https://wokwi.com/projects/373943840344172545



Procedimentos:

1. Utilize o circuito fornecido para ativar os leds conforme pedido. Neste circuito os leds estão ligados ao PORTD e o encoder (nossa interface de entrada) ao PORTC. Note que os pinos PC1 e PC2 estão ligados aos pinos DT e CLK do encoder (responsáveis pela codificação de movimento rotatório) e o pino SW (chave central do encoder), ao pino PC0.

2. Para entender como o encoder funciona, visite o site https://www.allaboutcircuits.com/projects/how-to-use-a-rotary-encoder-in-a-mcu-based-project/ ou o site https://howtomechatronics.com/tutorials/arduino/rotary-encoder-works-use-arduino/ .

3. Programe para que o circuito execute o seguinte: ao girar a chave no sentido anti-horário, os leds devem se acender da direita para a esquerda. Ao girar a chave no sentido horário, os leds se apagam da esquerda para a direita, um de cada vez. Ao clicar na chave central, os leds acesos se apagam e os apagados se acendem. Aqui a ideia é utilizar deslocamento de bits.

Notem que os pinos CLK e SW devem ter implementado em suas rotinas o debounce. Já o pino DT não necessita de debounce. Isto ocorre porquê existe uma defasagem entre os pinos CLK e DT em 90 graus. Desta forma, quando ocorre o acionamento da chave ligada ao CLK, a chave ligada ao DT já estará num estado estável, podendo a sua leitura ser efetuada sem qualquer controle temporal, simplesmente lendo o registrador PINC na posição 1.
Vocês podem aproveitar o código de debounce fornecido (aqui).

4. **EXTRA:** Ao segurar o botão central por mais de 900ms, todos os leds se apagam. Durante o intervalo em que o botão é seguro, o microcontrolador deve ficar livre para executar quaisquer outras rotinas.

5. Cole o código fonte do microcontrolador ao final deste arquivo. Anexe também, junto ao pdf, um arquivo texto com o código fonte.

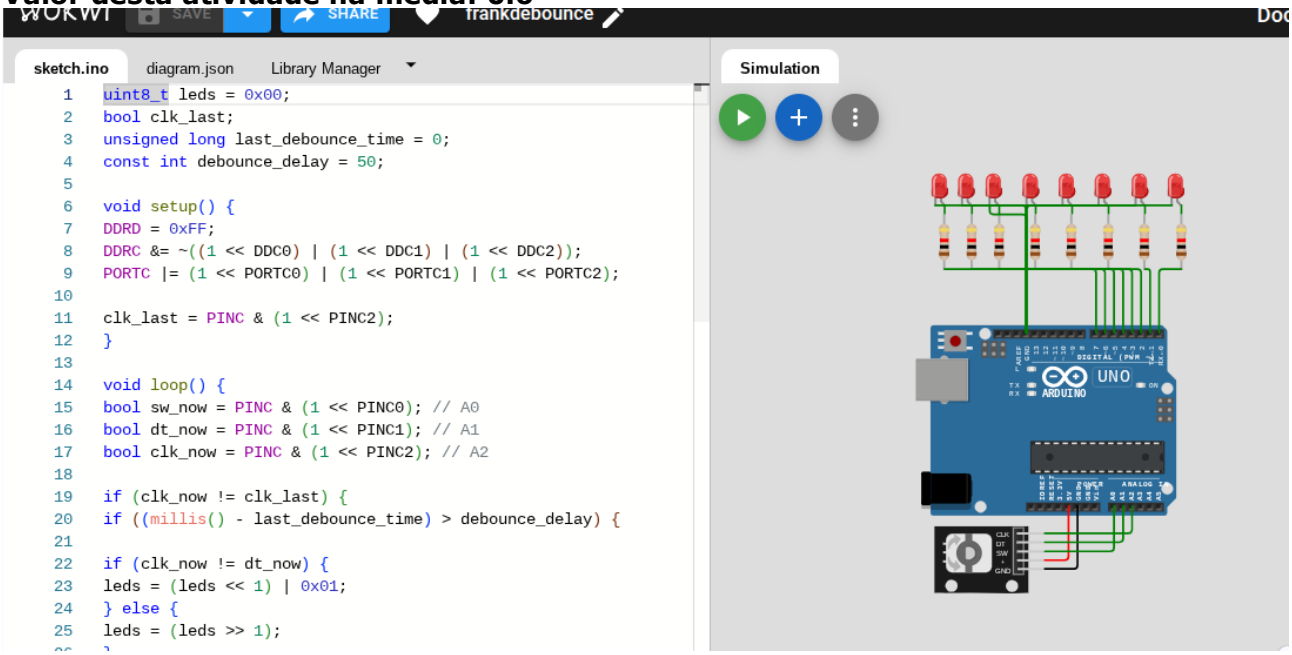
ATENÇÃO: Não utilize rotinas de interrupção. Não utilize pinMode, digitalWrite ou digitalRead. O uso destas funções acarretará na avaliação do trabalho com nota nula.

ATENÇÃO: O arquivo texto com o código fonte é parte essencial para a correção da atividade. Não anexar o fonte em arquivo separado pode resultar na avaliação com a nota zero.

Observação importante: O trabalho pode ser implementado também no Tinkercad caso o Wokwi esteja muito lento. No entanto, o Tinkercad não possui encoder rotativo. Para a implementação, utilize um botão para CLK e um interruptor deslizante para DT. Para SW utilize um botão tradicional também.

Rúbrica:
Encoder funciona: 30%
Debounce do encoder e sua chave: 30%
Leds fazem o pedido: 40%

Valor desta atividade na média: 0.6



EXTRA: 0.1 na média.

```
uint8_t leds = 0x00;
bool clk_last;
unsigned long last_debounce_time = 0;
const int debounce_delay = 50;

void setup() {
```

```
DDRD = 0xFF;
DDRC &= ~((1 << DDC0) | (1 << DDC1) | (1 << DDC2));
PORTC |= (1 << PORTC0) | (1 << PORTC1) | (1 << PORTC2);

clk_last = PINC & (1 << PINC2);
}

void loop() {
bool sw_now = PINC & (1 << PINC0); // A0
bool dt_now = PINC & (1 << PINC1); // A1
bool clk_now = PINC & (1 << PINC2); // A2

if (clk_now != clk_last) {
if ((millis() - last_debounce_time) > debounce_delay) {

if (clk_now != dt_now) {
leds = (leds << 1) | 0x01;
} else {
leds = (leds >> 1);
}

last_debounce_time = millis();
}
}
clk_last = clk_now;

static bool sw_last = HIGH;
if (sw_now == LOW && sw_last == HIGH) {
leds = ~leds;
delay(200);
}
sw_last = sw_now;
PORTD = leds;
}
```